

Программирование на C++

Контрольная работа №1

основы языка, указатели, функции

Общие требования

- Заведите отдельный репозиторий Git.
- Для каждого задания сделайте свою папку в репозитории.
- Используйте `.clang-format`.
- Контейнеры и строки стандартной библиотеки (`std::vector`, `std::string`, `std::map` и т.п.) использовать нельзя. Разрешены `cin` / `cout` и обычные массивы / указатели.
- В репозитории не должно быть лишних файлов IDE и артефактов сборки. Настройте `.gitignore`.

1. Дневник погоды

Вы ведёте дневник температуры. За неделю (или за N дней) записали по одному измерению на каждый день.

Что нужно сделать

Программа читает целое N , затем N целых чисел — температуры по Цельсию. Затем печатает три строки:

1. среднюю температуру (строго с одним знаком после запятой);
2. самую низкую и самую высокую температуру через пробел;
3. сколько дней температура была строго ниже нуля.

Пример

Ввод:

```
7
-2 1 3 0 -5 4 2
```

Вывод:

```
0.4
-5 4
2
```

Пояснение: сумма равна 3, среднее $3 / 7 = 0.4$. Минимум -5 , максимум 4. Ниже нуля были -2 и -5 — два дня. Ноль в «морозные» дни не входит.

Требования к реализации

- Температуры храните в массиве, выделенном через `new`.
- Утечки памяти недопустимы (`delete[]`).
- Среднее, минимум с максимумом и число морозных дней — три разные функции.
- Каждая функция получает массив (и при необходимости размер) и возвращает результат.
- В `main` только чтение, вызовы функций и печать.
- Если функция массив не меняет, параметры пометьте как `const`.
- Вещественный вывод — строго с одним знаком после запятой.

2. Копилка

У первокурсника есть копилка. В начале в ней 0 рублей. Он записывает операции в блокнот, пока не решит закончить день.

Что нужно сделать

Программа хранит текущую сумму (целое число, рубли). Читает команды по одной в строке, пока не встретит `stop`.

Команда	Значение
<code>deposit k</code>	положить k рублей (k — целое, $k > 0$)
<code>withdraw k</code>	взять k рублей ($k > 0$)
<code>balance</code>	напечатать текущую сумму
<code>reset</code>	обнулить копилку
<code>stop</code>	закончить работу

Если при снятии денег не хватает, сумму не меняйте, напечатайте `error` и читайте следующую команду. Если $k \leq 0$ для `deposit` / `withdraw` — тоже `error`. Неизвестная команда — `error`, сумма не меняется.

После `stop` ничего дополнительно печатать не нужно.

Пример

Ввод:

```
deposit 100
deposit 50
balance
withdraw 30
balance
withdraw 200
balance
reset
balance
stop
```

Вывод:

```
150
120
error
120
0
```

Требования к реализации

- Главное требование — указатели на функции.
- Каждую операцию оформите отдельной функцией одного и того же типа.
- Заведите таблицу команд: имя команды + указатель на функцию (два параллельных массива или массив пар).
- В `main`: прочитать имя команды; если `stop` — выйти; иначе найти имя в таблице и вызвать функцию по указателю.
- Цепочки `if (cmd == "deposit") ... else if ...` для выбора операции быть не должно.

- Сумму передавайте в функции по ссылке. Глобальную переменную для копилки не используйте.
- Добавьте ещё одну свою команду (например `double` — удвоить сумму, или `inc` — положить 1 рубль). Для неё — только новая функция и новая строка таблицы, цикл в `main` не меняйте.

3. Табло объявлений

В холле корпуса висит табло 5 строк на 10 символов. В начале все клетки — пробелы. Дежурный выводит короткие объявления и иногда стирает табло.

Что нужно сделать

Размер табло: 5 строк, 10 столбцов. Координаты: столбец `x` и строка `y`, оба с нуля. Левый верхний угол — `(0, 0)`.

Программа читает команды до `quit`:

Команда	Действие
<code>clear</code>	заполнить всё пробелами
<code>put x y c</code>	поставить символ <code>c</code> в клетку <code>(x, y)</code>
<code>text x y слово</code>	написать слово вправо начиная с <code>(x, y)</code> ; слово без пробелов, длина не больше 10
<code>print</code>	напечатать 5 строк по 10 символов
<code>quit</code>	выход

Если координаты или слово не помещаются на табло целиком, команда не меняет табло, программа печатает `error`.

- Для `put` клетка `(x, y)` должна быть внутри поля.
- Для `text` все буквы слова должны попасть в одну строку: `y` от 0 до 4 и `x + длина - 1 < 10`.

Пример

Ввод:

```
clear
text 0 0 Hello
put 9 4 !
print
text 8 0 TooLong
quit
```

Вывод команды `print` (пробелы в конце строк сохраняйте):

```
Hello
```

```
!
```

На команду `text 8 0 TooLong` — строка `error` (слово длины 7 с позиции 8 не влезает).

Требования к реализации

- Табло храните в двумерном массиве символов (или в массиве строк фиксированной длины).

- Операции вынесите в функции, например: `clear`, `put`, `text`, `print`. В цикле команд — чтение ввода и вызовы функций.
- `put` и `text` возвращают `false` при ошибке; тогда печатайте `error`.
- `text` рисует слово несколькими вызовами `put` или одним циклом по символам с той же проверкой границ. Логiku «вышли за край» не копируйте в трёх местах по-разному.

Подсказка

Сначала проверьте, что слово целиком влезает, и только потом пишите символы. Иначе при ошибке табло уже будет частично испорчено.

4. Работа с указателями

Нужно самостоятельно придумать и реализовать небольшую программу, в которой типы `int***` и `void*` действительно нужны, а не просто объявлены «для галочки».

Что нужно сдать

Одну программу на C++ с функцией `main` (другие функции заводить не нужно). В ней должны использоваться оба типа: `int***` и `void*`.

Программа решает понятную задачу (придумайте её). По коду и короткому комментарию в начале файла должно быть видно:

- зачем здесь три уровня указателей (`int***`);
- зачем здесь `void*`;
- что программа делает с данными.

Что не засчитывается

- Объявили `int***` и `void*`, напечатали адреса и вышли.
- Типы формально есть, но задача от них не зависит: те же действия легко сделать без `int***` / `void*`.
- «Искусственная» конструкция вида «указатель на указатель на указатель на одно число» без объяснения, зачем столько уровней.

Подсказка

- `int***` обычно появляется, когда есть «массив массивов массивов» целых или когда нужно изменить сам указатель на двумерную таблицу.
- `void*` — когда нужен указатель «на что-то неизвестное заранее»: общий буфер, параметр без фиксированного типа, разбор данных после приведения к `int*`, `char*` и т.п.
- В начале `main` напишите несколько строк комментария: какая задача, где какой тип и почему без него было бы хуже.